Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»
филиал «Минский радиотехнический колледж»

Учебный предмет
«Инструментальное программное обеспечение»

Отчет
по выполнению лабораторной работы №13 по теме
«Разработка, отладка и испытание программ с применением методов, статических методов и абстрактных методов»

Выполнила: учащаяся гр. 4к9392
Кривнюк Д. А.
Проверил: Лазовская Н. В.

Минск, 2025

Лабораторная работа №13

Тема работы: «Разработка, отладка и испытание программ с применением методов, статических методов и абстрактных методов»

1. Цель работы:

Научить использовать статические и абстрактные методы.

2. Задание:

В класс лабораторной работы № 11 необходимо добавить по два статических и абстрактных метода, реализовать перегрузку метода.

3. Оснащение работы:

Задание по варианту, ЭВМ, среда разработки **Python 3.7, IDLE**.

4. Порядок выполнения работы:

Код программы:

```
1. from abc import ABC, abstractmethod


class Car(ABC):
    def __init__(self, brand, color, mileage=0):
        self.brand = brand         self.color = color         self.mileage = mileage

    def info(self):
        return "Марка: " + self.brand + ", Цвет: " + self.color + ", Пробег: " + str(self.mileage) +
" км"

    def __str__(self):
        return self.info()

    def __add__(self, distance):
        if isinstance(distance, (int, float)):
            self.mileage += distance             return self         return NotImplemented

    def drive(self, distance):
        self.mileage += distance         return "Машина проехала " + str(distance) + " км. Общий
пробег: " + str(self.mileage) + " км"

    def is_old(self):
        if self.mileage > 100000:
            return "У машины большой пробег."         else:
            return "У машины небольшой пробег."

    @staticmethod     def convert_km_to_miles(km):
        return km * 0.621371

    @staticmethod     def get_vehicle_type():
        return "Автомобиль"

    @abstractmethod     def get_engine_type(self):
        pass

    @abstractmethod     def calculate_tax(self):
        pass


class PassengerCar(Car):
    def __init__(self, brand, color, mileage=0, engine_type="Бензиновый"):
        super().__init__(brand, color, mileage)
```

```
        self.engine_type = engine_type

    def get_engine_type(self):
        return f"Тип двигателя: {self.engine_type}"

    def calculate_tax(self):

        base_tax = 5000         if self.mileage > 150000:
            return base_tax * 0.8         return base_tax


    def info(self):
        return super().info() + f", Тип двигателя: {self.engine_type}"


print("Введите данные о машине:")
brand = input("Марка: ")
color = input("Цвет: ")
mileage = int(input("Пробег (в км): "))
engine_type = input("Тип двигателя (Бензиновый/Дизельный/Электрический): ")

my_car = PassengerCar(brand, color, mileage, engine_type)

print("\nИнформация о вашей машине:")
print(my_car.info())

print(f"\nПробег в милях: {Car.convert_km_to_miles(my_car.mileage):.2f}")
print(f"Тип транспортного средства: {Car.get_vehicle_type()}")

print(f"\n{my_car.get_engine_type()}")
print(f"Налог на транспорт: {my_car.calculate_tax()} руб.")

print("\n--- Демонстрация перегрузки операторов ---")
print("Использование __str__:", my_car)

my_car + 500 print("После использования оператора +500 км:", my_car)

while True:
    print("\nВыберите действие:")
    print("1 - Проехать расстояние")
    print("2 - Проверить пробег")
    print("3 - Показать информацию")
    print("4 - Конвертировать пробег в мили")
    print("5 - Рассчитать налог")
    print("0 - Выйти")

    choice = input("Ваш выбор: ")

    if choice == "1":
        distance = int(input("Сколько км проехать? "))
        print(my_car.drive(distance))
    elif choice == "2":
        print(my_car.is_old())
    elif choice == "3":
        print(my_car.info())
    elif choice == "4":
        miles = Car.convert_km_to_miles(my_car.mileage)
        print(f"Пробег {my_car.mileage} км = {miles:.2f} миль")
    elif choice == "5":
        tax = my_car.calculate_tax()
        print(f"Налог на транспорт: {tax} руб.")
    elif choice == "0":
        print("Выход из программы.")
        break     else:
        print("Неверный выбор, попробуйте снова.")
2.
```

Результат выполнения программы представлен на рисунке 1:

```
Run    ЛАБА12
C:\Users\HONOR\PycharmProjects\PythonProject4\.venv\Scripts\python.exe C:\Users\HONOR\PycharmProjects\PythonProject4\ЛАБА1
Введите данные о машине:
Марка: BMW
Цвет: чорно
Пробег (в км): 100000
Тип двигателя (Бензиновый/Дизельный/Электрический): Бензиновый

Информация о вашей машине:
Марка: BMW, Цвет: чорно, Пробег: 100000 км, Тип двигателя: Бензиновый

Пробег в милях: 62137.10
Тип транспортного средства: Автомобиль

Тип двигателя: Бензиновый
Налог на транспорт: 5000 руб.

--- Демонстрация перегрузки операторов ---
Использование __str__: Марка: BMW, Цвет: чорно, Пробег: 100000 км, Тип двигателя: Бензиновый
После использования оператора +500 км: Марка: BMW, Цвет: чорно, Пробег: 100500 км, Тип двигателя: Бензиновый

Выберите действие:
1 - Проехать расстояние
2 - Проверить пробег
3 - Показать информацию
4 - Конвертировать пробег в мили
5 - Рассчитать налог
0 - Выйти
Ваш выбор:
```

Рисунок 1 – Результат решение задания

5.Контрольные вопросы и задания:

1. Для чего используют методы?

Методы в программировании нужны для того, чтобы описывать поведение объектов и работать с их данными. С помощью методов можно изменять значения полей объекта, выполнять вычисления, проверять условия или возвращать информацию.

2. В чем особенность статических и абстрактных методов?

Статические методы принадлежат классу, а не конкретному объекту. Их можно вызывать без создания экземпляра, и они не имеют доступа к данным объекта (self). Обычно такие методы используют для вспомогательных операций – например, проверки корректности введённых данных или форматирования строк. Абстрактные методы, наоборот, задают обязательный интерфейс для всех классов-наследников. Они объявляются в абстрактном классе без реализации, и любой класс, который наследуется от него, обязан их реализовать. Это гарантирует, что все объекты определённого типа будут иметь одинаковый набор ключевых функций, даже если их внутренняя реализация отличается.